1. (2 Punkte) Liste ist unser selbstgebastelter Datentyp. Wir nummerieren die Einträge der Liste mit Zeiger `anf` an der Stelle 0.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Gib den Index des Elements an, auf das der Zeiger `pos` zeigt.

```
a = Liste()
a.insert(1)
a.insert(2)
a.advance()
a.insert(3)
a.insert(4)
a.reset()
a.advance()
a.insert(6)
```

Lösung: a. `2 6 4 3 1` b. `1`

2. (2 Punkte) Liste ist unser selbstgebastelter Datentyp. Wir nummerieren die Einträge der Liste mit Zeiger `anf` an der Stelle 0.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Gib den Index des Elements an, auf das der Zeiger `pos` zeigt.

```
a = Liste()
a.insert(3)
a.advance()
a.insert(2)
a.insert(5)
a.insert(3)
a.reset()
a.advance()
a.delete()
a.insert(4)
a.advance()
```

Lösung: a. `3 4 5 2` b. `2`

3. (3 Punkte) Liste ist unser selbstgebastelter Datentyp.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Welchen Inhalt hat das aktuelle Element?

```
a = Liste()
a.insert(4)
a.insert(3)
a.advance()
a.insert(6)
a.reset()
a.advance()
a.delete()
a.insert(9)
a.advance()
a.advance()
a.insert(7)
a.reset()
a.advance()
a.advance()
```

Lösung: a. `3 9 4 7` b. `4`

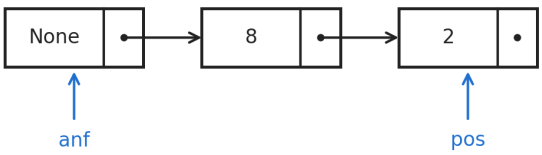
4. (3 Punkte) Liste ist unser selbstgebastelter Datentyp.

```
a = Liste()
a.insert(5)
a.insert(8)
a.advance()
a.insert(2)
a.advance()
a.delete()
```

   (a) (2 Punkte) Zeichne die Einträge der Liste a mit den Zeigern `anf` und `pos` nach diesen Anweisungen.

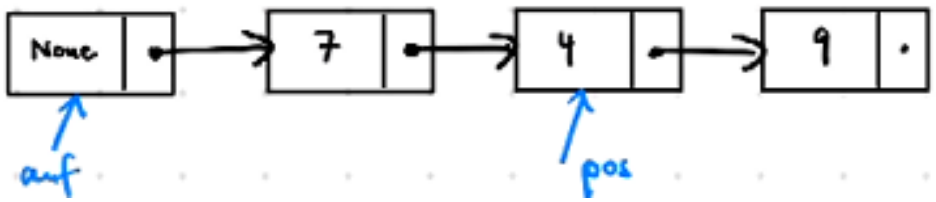
   (b) (1 Punkt) Was liefert jetzt `a.endpos()`?

Lösung:



b) True, denn der Eintrag, auf den pos zeigt, hat keinen Nachfolger.

5. (5 Punkte) Liste ist unser selbstgebastelter Datentyp. Das Bild zeigt die Situation der Liste a.



a. Was liefern die folgenden Anweisungen:

```
print(a.elem())
print(a.endpos())
```

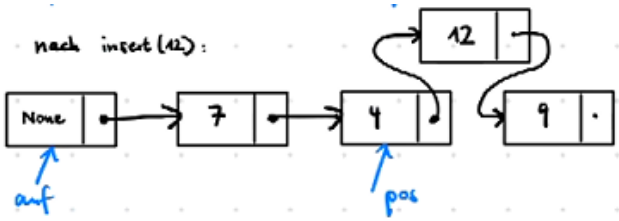
b. Zeichne die Situation nach der Ausführung von a.insert(12)
c. Was liefern jetzt die folgenden Anweisungen:

```
print(a.elem())
print(a.endpos())
```

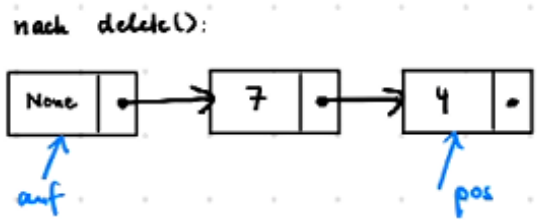
d. Wir gehen nochmal von der Anfangssituation aus. Zeichne die Situation nach Ausführung von a.delete()
e. Was liefern jetzt die folgenden Anweisungen:

```
print(a.endpos())
print(a.elem())
```

Lösung: a. 9, False
b.

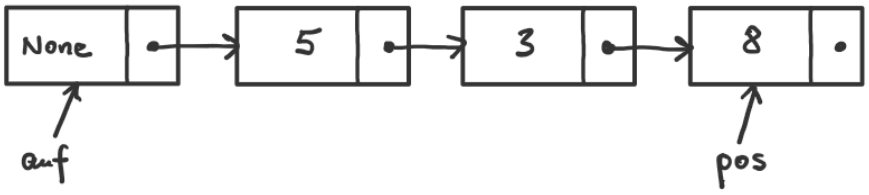


c. 12, False
d.



e. True, ein Fehler wird geworfen

6. (4 Punkte) Liste ist unser selbstgebastelter Datentyp. Das Bild zeigt die Situation der Liste a.



a. Was liefern die folgenden Anweisungen:

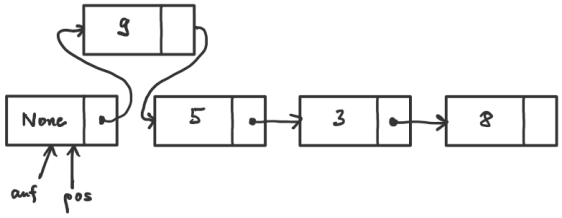
```
print(a.endpos())
print(a.elem())
```

b. Es werden folgende Anweisungen ausgeführt:::

```
a.reset()
a.insert(9)
```

Zeichne die entstandene Situation.
c. Was liefert jetzt die Anweisung a.elem()???

Lösung: a. True, ein Fehler wird geworfen
b.



c. 9

7. (3 Punkte) Die Liste a enthält die Zahlen 3 7 5. Die Schleife soll alle Elemente ausgeben.

```
a.reset()
while not a.endpos():
    a.advance()
    print(a.elem())
```

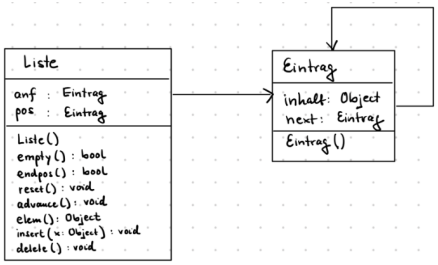
(a) (2 Punkte) Was geschieht stattdessen?

(b) (1 Punkt) Korrigiere die Schleife.

Lösung: a) Ausgegeben werden 7 und 5, die 3 wird übersprungen. Nach der 5 ist das Ende erreicht und `a.elem()` wirft einen RuntimeError.
b) Die beiden Zeilen im Schleifenrumpf werden getauscht:

```
a.reset()
while not a.endpos():
    print(a.elem())
    a.advance()
```

8. (4 Punkte) Das Bild zeigt das UML-Diagramm unseres Datentyps Liste. Implementiere in Python die Klassen Eintrag und die Klasse Liste (nur Konstruktor und Methode delete). Achte dabei auf mögliche Fehlersituationen.



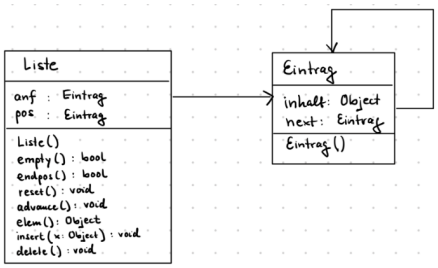
Lösung:

```
class Eintrag:
    def __init__(self):
        self.inhalt = None
        self.next = None

class Liste:
    def __init__(self):
        self.anf = Eintrag()
        self.pos = self.anf

    def delete(self):
        if self.endpos(): raise RuntimeError("Fehler: Liste am Ende")
        self.pos.next = self.pos.next.next
```

9. (4 Punkte) Das Bild zeigt das UML-Diagramm unseres Datentyps Liste. Implementiere in Python die Klassen Eintrag und die Klasse Liste (nur Konstruktor und die Methoden endpos und advance). Achte dabei auf mögliche Fehlersituationen.



Lösung:

```
class Eintrag:
    def __init__(self):
        self.inhalt = None
        self.next = None

class Liste:
    def __init__(self):
        self.anf = Eintrag()
        self.pos = self.anf

    def endpos(self):
        return self.pos.next is None

    def advance(self):
        if self.endpos(): raise RuntimeError("Fehler: Liste am Ende")
        self.pos = self.pos.next
```

10. (4 Punkte) Die Klasse Eintrag und der Konstruktor unserer Klasse Liste sind gegeben.

```
class Eintrag:
    def __init__(self):
        self.inhalt = None
        self.next = None

class Liste:
    def __init__(self):
        self.anf = Eintrag()
        self.pos = self.anf
```

Implementiere die Methoden `elem` und `insert`. Achte dabei auf mögliche Fehlersituationen.

Lösung:

```
    def elem(self):
        if self.endpos(): raise RuntimeError("Fehler: Liste am Ende")
        return self.pos.next.inhalt

    def insert(self, x):
        hilf = Eintrag()
        hilf.inhalt = x
        hilf.next = self.pos.next
        self.pos.next = hilf
```

11. (3 Punkte) In der Methode `insert` sind zwei Zeilen vertauscht.

```
def insert(self, x):
    hilf = Eintrag()
    hilf.inhalt = x
    self.pos.next = hilf
    hilf.next = self.pos.next
```

(a) (1 Punkt) Welche Zeilen müssen getauscht werden?

(b) (2 Punkte) Was passiert mit dieser falschen Methode, wenn die folgenden Anweisungen ausgeführt werden? Begründe.

```
a = Liste()
a.insert(1)
a.insert(2)
while not a.endpos():
    print(a.elem())
    a.advance()
```

Lösung: a) Die letzten beiden Zeilen. Der neue Eintrag muss zuerst den Nachfolger übernehmen.
b) Die Zahl 2 wird endlos ausgegeben. `self.pos.next` ist bereits `hilf`, also zeigt `hilf.next` auf `hilf` selbst. Der neue Eintrag ist sein eigener Nachfolger, `endpos` wird nie True und der bisherige Eintrag mit der 1 ist nicht mehr erreichbar.

12. (6 Punkte) Liste ist unser selbstgebastelter Datentyp. Benutze nur die Methoden der Liste.
    (a) (2 Punkte) Erzeuge eine Liste a mit den Zahlen von 1 bis 100 in aufsteigender Reihenfolge.
    (b) (2 Punkte) Berechne die Summe der Elemente von a.
    (c) (2 Punkte) Verschiebe alle durch 3 teilbaren Zahlen aus a in eine neue Liste b. Die Reihenfolge der Zahlen soll erhalten bleiben. Danach ist die Summe der Elemente von a gleich 3367 und die von b gleich 1683.

Lösung:

```
a = Liste()
for i in range(1, 101):
    a.insert(i)
    a.advance()

summe = 0
a.reset()
while not a.endpos():
    summe += a.elem()
    a.advance()

b = Liste()
a.reset()
while not a.endpos():
    x = a.elem()
    if x % 3 == 0:
        b.insert(x)
        b.advance()
        a.delete()
    else:
        a.advance()
```

13. (3 Punkte) Die Funktion `anzahl(a, x)` bekommt eine Liste a (unser Datentyp) und gibt zurück, wie oft x in der Liste vorkommt. Benutze nur die Methoden der Liste. Implementiere die Funktion.

Lösung:

```
def anzahl(a, x):
    n = 0
    a.reset()
    while not a.endpos():
        if a.elem() == x:
            n += 1
        a.advance()
    return n
```

14. (3 Punkte) Die Funktion `ersetze(a, alt, neu)` ersetzt in der Liste a (unser Datentyp) jedes Vorkommen von alt durch neu. Die Reihenfolge der Elemente bleibt erhalten. Benutze nur die Methoden der Liste. Implementiere die Funktion.

Lösung:

```
def ersetze(a, alt, neu):
    a.reset()
    while not a.endpos():
        if a.elem() == alt:
            a.delete()
            a.insert(neu)
        a.advance()
```

15. (4 Punkte) Die Funktion `doppelte_entfernen(a)` löscht aus der sortierten Liste a (unser Datentyp) alle doppelten Elemente, sodass jeder Wert nur noch einmal vorkommt. Aus 1 1 2 3 3 3 5 wird 1 2 3 5. Die Liste kann leer sein. Benutze nur die Methoden der Liste. Implementiere die Funktion.

Lösung:

```
def doppelte_entfernen(a):
    a.reset()
    if a.endpos():
        return
    letztes = a.elem()
    a.advance()
    while not a.endpos():
        if a.elem() == letztes:
            a.delete()
        else:
            letztes = a.elem()
            a.advance()
```

16. (4 Punkte) Die Klasse `FeatureListe` erbt von Liste. Sie hat zusätzlich ein Attribut `gerade`, das genau dann `True` ist, wenn die Anzahl der gespeicherten Listenelemente gerade ist. Schreibe die Klasse `FeatureListe`.

Lösung:

```
class FeatureListe(Liste):
    def __init__(self):
        super().__init__()
        self.gerade = True

    def insert(self, x):
        super().insert(x)
        self.gerade = not self.gerade

    def delete(self):
        super().delete()
        self.gerade = not self.gerade
```

17. (4 Punkte) Die Klasse `FeatureListe` erbt von Liste und hat die zusätzliche Methode `laenge`, die die Anzahl der gespeicherten Elemente zurückgibt. Das aktuelle Element soll sich dabei nicht ändern. Schreibe die Klasse `FeatureListe`.

Lösung:

```
class FeatureListe(Liste):
    def laenge(self):
        n = 0
        e = self.anf.next
        while e is not None:
            n += 1
            e = e.next
        return n
```

18. (3 Punkte) a. Gib für die beteiligten Verweisboxen die Attributwerte nach Ablauf der untenstehenden Anweisungen an. (Angabe in der Form: inhalt = 'e', oben = a, unten = b)
b. Welche Ausgabe erscheint auf der Konsole?

```
class VerweisBox:
    def __init__(self, inhalt, unten=None, oben=None):
        self.inhalt = inhalt
        self.unten = unten
        self.oben = oben
        if unten is not None: self.unten.oben = self
        if oben  is not None: self.oben.unten = self

    def __str__(self):
        return self.inhalt

a = VerweisBox('a')
b = VerweisBox('b')
c = VerweisBox('c',a,b)
d = VerweisBox('d',b,c)
print(a.unten, b.unten.oben, b.oben.oben.oben, c.unten, c.oben.oben.unten.unten.unten)
```

Lösung:

```
Die Werte der Attribute:
inhalt = 'a', unten = None, oben = c
inhalt = 'b', unten = c, oben = d
inhalt = 'c', unten = d, oben = b
inhalt = 'd', unten = b, oben = c

Die print-Ausgabe:
None b b d d
```

19. (3 Punkte) Welche Ausgabe erscheint auf der Konsole?

```
class VerweisBox:
    def __init__(self, inhalt, unten=None, oben=None):
        self.inhalt = inhalt
        self.unten = unten
        self.oben = oben
        if unten is not None: self.unten.oben = self
        if oben  is not None: self.oben.unten = self

    def __str__(self):
        return self.inhalt

a = VerweisBox('a')
b = VerweisBox('b')
c = VerweisBox('c',a,b)
d = VerweisBox('d',c,a)
print(c.unten.oben)
print(b.unten.oben.unten.unten)
print(d.unten.unten.unten)
```

Lösung: c, a, d

20. (3 Punkte) Eine Python-Liste ist ein Array. Unsere Liste besteht aus verketteten Einträgen. Entscheide und begründe kurz.
    (a) (1 Punkt) Ein Programm greift sehr oft über die Position i auf ein Element zu. Welche Datenstruktur passt besser?
    (b) (1 Punkt) Ein Programm geht durch eine lange Warteliste und fügt dabei oft Elemente ein oder entfernt sie. Welche Datenstruktur passt besser?
    (c) (1 Punkt) Warum lohnt sich die binäre Suche auf unserer Liste nicht, auch wenn sie sortiert ist?

Lösung: a) Das Array. Der Zugriff über die Position braucht dort O(1), bei unserer Liste O(n).
b) Die verkettete Liste. Einfügen und Löschen an der aktuellen Stelle brauchen O(1), beim Array müssen alle Elemente dahinter verschoben werden.
c) Die binäre Suche springt in die Mitte. Bei der verketteten Liste müsste man sich dorthin durchhangeln, jeder dieser Sprünge kostet O(n).

21. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) (1 Punkt) Was ist ein Abstrakter Datentyp?
    (b) (1 Punkt) Warum zeigt `pos` auf den Eintrag vor dem aktuellen Element und nicht auf das aktuelle Element selbst?
    (c) (1 Punkt) Wozu dient der Dummy-Eintrag am Anfang der Liste?
    (d) (1 Punkt) Warum nennt man die Liste eine rekursive Datenstruktur?

Lösung: a) Ein ADT besteht aus einer Datenstruktur und Operationen darauf. Festgelegt ist nur die Schnittstelle, nicht die Implementierung.
b) Beim Einfügen und Löschen muss der next-Zeiger des Vorgängers geändert werden. Vom aktuellen Element aus kommen wir nicht zurück zum Vorgänger.
c) Dadurch hat auch das erste Element einen Vorgänger. Einfügen und Löschen am Anfang brauchen keinen Sonderfall.
d) Die Klasse Eintrag hat mit next ein Attribut vom Typ Eintrag. Sie verweist also auf sich selbst (rekursive Assoziation).